



KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM

Chương 5: Kiến trúc và mẫu thiết kế

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

Nội dung của chương này tập trung xem xét một số chủ đề sau đây:

- Khái niệm mẫu thiết kế
- Định dạng mẫu thiết kế
- Phân loại và sử dụng mẫu thiết kế
- Mẫu thiết kế cơ bản: Lớp Interface và Abstract

| 5.1 KHÁI NIỆM MẪU THIẾT KẾ

- Mẫu thiết kế (**design pattern**) là một giải pháp đã được áp dụng rộng rãi để giải quyết những vấn đề thường hay xảy ra giống nhau trong thực tế.
- Khái niệm mẫu thiết kế lần đầu tiên được đề xuất trong ngành xây dựng bởi kiến trúc sư Christopher Alexander vào những năm 1970. Ông cho rằng mẫu thiết kế là một giải pháp hiệu quả để giải quyết nhiều vấn đề thường hay lặp đi lặp lại trong thực tế kiến trúc.
- Năm 1987, Kent Beck và War Cunningham đã trình bày áp dụng những ý tưởng của Alexander cho thiết kế và phát triển phần mềm đặc biệt dành cho lập trình hướng đối tượng.
- Đến năm 1994, cuốn sách “Design Patterns: Elements of Reusable Object-Oriented Software” của Gang of Four (GOF) - Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides – với đề xuất 23 mẫu thiết kế đã lan rộng và ảnh hưởng mạnh mẽ ứng dụng của các mẫu thiết kế.

I KHÁI NIỆM MẪU THIẾT KẾ

- Mẫu thiết kế là một cách hiệu quả để truyền tải những những thiết kế có chất lượng. Nó có thể là từ vựng của một phần tử thiết kế để giải quyết vấn đề đang gặp phải hay là một ngôn ngữ chung để trao đổi kinh nghiệm.
- Mẫu thiết kế không phải là một phát minh mà đúng ra là một phần tài liệu bàn về cách giải quyết một vấn đề được xem là hữu hiệu nhất qua kinh nghiệm xây dựng nhiều hệ thống phần mềm.
- Không phải cái gì tốt nhất trong thực tế đều được xem là mẫu mà phải thỏa mãn Quy luật *“ít nhất ba”*. Luật này cho rằng mẫu đó *phải được chứng tỏ là lặp đi lặp lại và là tốt nhất trong ít nhất ba hệ thống đã xây dựng*. Ý nghĩa của luật này là nhằm đảm bảo rằng đã có một số chuyên gia phần mềm ứng dụng giải pháp đề xuất để giải quyết một số vấn đề trong thiết kế phần mềm rồi.

I KHÁI NIỆM MẪU THIẾT KẾ

- Các mẫu thiết kế không cung cấp các giải pháp cho mọi vấn đề trong thiết kế và phát triển phần mềm thực tế mà chỉ đưa ra các giải pháp tái sử dụng để giải quyết các vấn đề phát triển phần mềm thường gặp trong một bối cảnh cụ thể nào đó.
- Điều này có nghĩa rằng nó chỉ là các mô hình cung cấp các giải pháp tốt nhất cho một vấn đề trong một ngữ cảnh cụ thể nào đó nhưng không thể hiệu quả trong ngữ cảnh khác. Vì vậy, các giải pháp của các mẫu thiết kế được đề xuất có thể áp dụng trong ngữ cảnh này nhưng không áp dụng được trong một số ngữ cảnh khác.

| Mẫu thiết kế và Framework

- Mẫu thiết kế không tồn tại dưới dạng thành phần phần mềm mà phải được cài đặt tại thời điểm sử dụng, không nên nhầm lẫn giữa framework với mẫu thiết kế.
- Framework là một dạng phần mềm gồm các thành phần phối hợp hoạt động với nhau để cung cấp một kiến trúc cho một miền ứng dụng nên nó có thể cài đặt nhiều mẫu thiết kế.

| Mẫu thiết kế và Framework

Mẫu thiết kế	Framework
Mẫu thiết kế là giải pháp lặp đi lặp lại đối với vấn đề nảy sinh trong vòng đời của ứng dụng phần mềm	Framework là nhóm thành phần cộng hợp với nhau để cung cấp một kiến trúc có thể sử dụng lại cho những ứng dụng
Mục đích là: + Giúp cải tiến chất lượng phần mềm (sử dụng lại, bảo trì, mở rộng...) + Giảm thời gian phát triển	Mục đích là: + Giúp cải tiến chất lượng phần mềm (sử dụng lại, bảo trì, mở rộng...) + Giảm thời gian phát triển
Mẫu là một cấu trúc logic	Framework là một dạng phần mềm
Mô tả mẫu thường độc lập với ngôn ngữ lập trình và chi tiết cài đặt	Framework đặc trưng cho cài đặt
Mẫu là tổng quát hơn và có thể sử dụng trong nhiều ứng dụng	Framework cung cấp chức năng cho miền cụ thể
Mẫu thiết kế không tồn tại dưới dạng thành phần phần mềm. Nó cần được cài đặt mỗi khi sử dụng	Framework không phải là một ứng dụng đầy đủ, các ứng dụng có thể xây dựng bằng cách kế thừa từ các thành phần có sẵn
Mẫu thiết kế là cách thiết kế tốt và được sử dụng để thiết kế các framework	Mẫu thiết kế có thể sử dụng trong thiết kế và cài đặt các framework nghĩa là framework có thể chứa nhiều mẫu thiết kế

I 5.1 ĐỊNH DẠNG MẪU THIẾT KẾ

- Vì các mẫu thiết kế được sử dụng bởi nhiều người nên chúng cần được thể hiện theo một định dạng được chấp nhận rộng rãi.
- Một số đặc trưng của các mẫu thiết kế:
 - **Tên mẫu:** Tên viết ngắn gọn của mẫu thường là một hoặc hai từ. Ví dụ, tên mẫu phải chọn làm sao thể hiện được mục đích của mô hình và được sử dụng như một tên duy nhất đặc trưng cho mẫu đó.
 - **Phân loại:** Các mẫu được phân chia thành nhiều loại khác nhau.
 - Nhóm mẫu tạo dựng (creational) là các mẫu tập trung vào cách tạo các đối tượng;
 - Nhóm mẫu cấu trúc (structural) tập trung vào việc sắp xếp các đối tượng với nhau thành một cấu trúc tổng thể lớn hơn;
 - Nhóm mẫu hành vi (behavioral) liên quan đến hành vi, cộng tác giữa các đối tượng để thực hiện một mục tiêu nào đó.
 - **Mục đích:** Một mô tả chung ngắn gọn (một hoặc hai câu) về mô hình. Ví dụ, mẫu Observer mô tả phụ thuộc 1-nhiều giữa các đối tượng để khi một đối tượng thay đổi trạng thái thì tất cả phụ thuộc của nó có thể nhận biết và cập nhật tự động.

| Một số đặc trưng của các mẫu thiết kế

- **Định danh:** Các mẫu đều có định danh của mẫu đó.
- **Động lực:** Xác định vấn đề mà mẫu thiết kế này có thể được sử dụng để giải quyết.
- **Khả năng áp dụng:** Lĩnh vực mà mẫu thiết kế này có thể được sử dụng.
- **Cấu trúc:** Những biểu đồ lớp và thành phần với UML nhằm thể hiện cách hoạt động mẫu thiết kế này.
- **Các bên tham gia:** Xác định các đối tượng liên quan hoặc những công việc mà đối tượng đó được yêu cầu thực hiện.
- **Cộng tác:** Xác định sự cộng tác của các bên tham gia.
- **Kết quả:** Lợi ích và hạn chế của mẫu thiết kế này (thêm những gợi ý để giải quyết các vấn đề về hạn chế).
- **Cài đặt:** Những gợi ý, chỉ dẫn về cách sử dụng mẫu này bao gồm cả những thủ thuật, các cách ứng dụng hiệu quả và những điều nên tránh.

| Một số đặc trưng của các mẫu thiết kế

- **Mã nguồn mẫu:** Có ví dụ triển khai đầy đủ cho mẫu này được viết dưới dạng ngôn ngữ C++, java, C#...
- **Các áp dụng:** Ví dụ về các lĩnh vực mà mẫu này đã được áp dụng trong thực tế.
- **Các mẫu liên quan:** Các mẫu thiết kế tương tự như mẫu này hoặc cũng mang lại hiệu quả tương tự khi sử dụng như mẫu hiện thời.

| 5.3 PHÂN LOẠI MẪU THIẾT KẾ

- Hệ thống 23 mẫu thiết kế của GoF được chia thành ba nhóm: Nhóm tạo dựng, nhóm cấu trúc và nhóm hành vi.
- Nhóm mẫu tạo dựng:
 - Mục đích của mẫu nhóm này nhằm giải quyết công việc thường xuyên là tạo ra các đối tượng.
 - Các mẫu sẽ tạo ra một cơ chế đơn giản, thống nhất khi tạo các thể hiện của đối tượng.
 - Cho phép đóng gói các chi tiết về các lớp nào được khởi tạo và cách các thể hiện này được tạo ra.
 - Nó khuyến khích sử dụng **interface** nhằm giảm độ liên kết.
 - Nhóm này gồm các mẫu thiết kế sau đây: **Abstract Factory**, **Factory Method**, **Builder**, **Prototype**, và **Singleton**.

| PHÂN LOẠI MẪU THIẾT KẾ

- Nhóm mẫu cấu trúc:

- Nhóm này chủ yếu giải quyết vấn đề một đối tượng ủy thác trách nhiệm cho những đối tượng khác. Điều này dẫn đến kiến trúc phân tầng của các thành phần với độ kết nối thấp.
- Tạo điều kiện giao tiếp giữa các đối tượng khi một đối tượng không thể truy nhập đối tượng khác theo cách thông thường hay khi một đối tượng không thể sử dụng được do giao diện không tương thích.
- Cung cấp các cách để cấu trúc một đối tượng với quan hệ hợp thành được tạo ra đầy đủ. Nhóm này liên quan tới các quan hệ cấu trúc giữa các thể hiện, bằng cách sử dụng các quan hệ kế thừa, liên kết, phụ thuộc. Để xem thông tin về mẫu này phải dựa vào **biểu đồ lớp** của mẫu.
- Nhóm này gồm các mẫu sau đây: **Adapter**, **Bridge**, **Composite**, **Decorator**, **Facade**, **Proxy** và **Flyweight**.

| PHÂN LOẠI MẪU THIẾT KẾ

- Nhóm mẫu hành vi :

- Nhóm mẫu thiết kế này liên quan đến các quan hệ gán trách nhiệm để cung cấp các chức năng giữa các đối tượng trong hệ thống.
- Mô tả cơ chế giao tiếp giữa các đối tượng và xác định cơ chế chọn các thuật toán khác nhau bởi các đối tượng khác nhau tại thời gian chạy.
- Đối với các mẫu thuộc nhóm này ta có thể dựa vào **biểu đồ giao tiếp** và **biểu đồ tuần tự** để giải thích cách chuyển giao của các chức năng.
- Nhóm này gồm có một số mẫu sau đây: **Interpreter**, **Template Method**, **Chain of Responsibility**, **Command**, **Iterator**, **Mediator**, **Memento**, **Observer**, **State**, **Strategy** và **Visitor**

I 5.4 SỬ DỤNG MẪU THIẾT KẾ

5.4.1 Khi nào sử dụng mẫu thiết kế?

- *Mẫu thiết kế được hiểu theo nghĩa tái sử dụng ý tưởng hơn là tái sử dụng mã lệnh.* Mẫu thiết kế cho phép các nhà thiết kế ngồi lại cùng nhau để giải quyết một vấn đề nào đó mà không phải mất nhiều thời gian tranh cãi.
- *Mẫu thiết kế hỗ trợ tái sử dụng kiến trúc và mô hình thiết kế phần mềm quy mô lớn.* Cần phân biệt mẫu thiết kế với framework. Framework hỗ trợ tái sử dụng mô hình thiết kế và mã nguồn ở mức chi tiết hơn. Trong khi đó, mẫu thiết kế được vận dụng ở mức tổng quát hơn.
- *Mẫu thiết kế là đa tương thích* nghĩa là nó không phụ thuộc vào ngôn ngữ lập trình, công nghệ hoặc các nền tảng lớn như J2EE của Sun hay Microsoft .NET. Các kiến trúc dựa trên mẫu thiết kế đã được sử dụng khá nhiều trong các phần mềm mã nguồn mở, trong nền tảng J2EE hoặc .NET.

I 5.4 SỬ DỤNG MẪU THIẾT KẾ

5.4.2 Sử dụng mẫu thiết kế như thế nào?

Các mẫu thiết kế cần phải thay đổi để phù hợp với tình huống sử dụng cụ thể. Sau đây là một số lý do vì sao cần sự điều chỉnh:

- **Sự khác biệt về ngôn ngữ biên dịch:** Các ngôn ngữ lập trình kể cả các ngôn ngữ hướng đối tượng đều có những khác biệt nhau như cú pháp, biên dịch...Do đó, các mẫu thiết kế cần phải hiệu chỉnh sao cho phù hợp với ngôn ngữ đang sử dụng cho dự án.
- **Sự khác biệt về quan điểm:** Thông thường, có thể có nhiều giải pháp đối với mỗi vấn đề đặt ra trong khi thiết kế. Do đó, một mẫu thiết kế được đưa ra bởi một chuyên gia hoặc một nhóm các chuyên gia không có nghĩa là nó đã là hoàn hảo.
- **Sự khác biệt về kiểu:** Thiết kế và kiểu mã hóa có thể nảy sinh từ nhiều nguồn như kinh nghiệm, nguyên tắc viết mã của doanh nghiệp, các yêu cầu của một thư viện hoặc framework. Nếu một mô hình không phù hợp với kiểu mà dự án sử dụng thì chúng ta phải điều chỉnh nó.

I SỬ DỤNG MẪU THIẾT KẾ

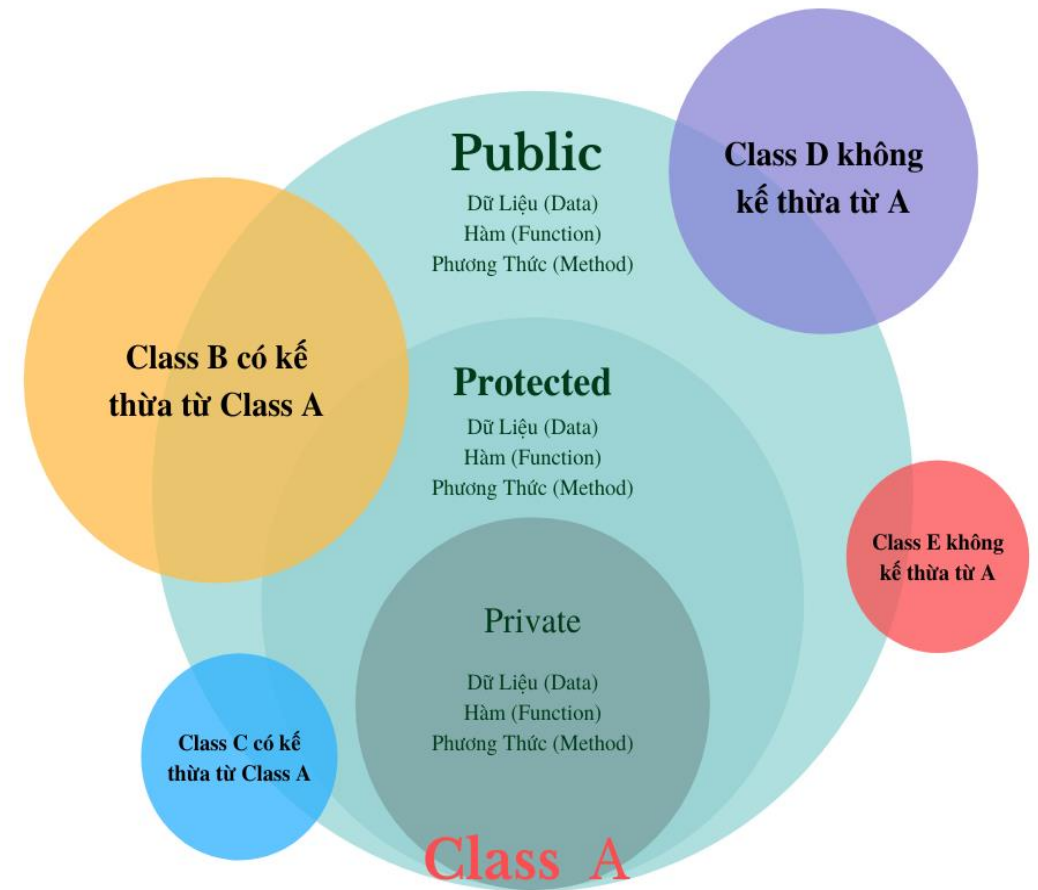
- **Vấn đề khác nhau:** Mỗi một mẫu thiết kế phải được thể hiện hết sức tổng quát nên một số mô hình lớp trình bày trong mẫu có thể sẽ không được sử dụng.
- **Thành phần, kế thừa và đa kế thừa:** Các ngôn ngữ hướng đối tượng và các nhà phát triển thường có sự khác biệt trong cách sử dụng kế thừa và thành phần. Ví dụ, trong C++ có đa kế thừa nhưng Java thì lại không có.
- **Đơn giản hóa:** Một mẫu thiết kế thường là tổng quát hóa cho nhiều tính năng và do đó có thể sẽ phức tạp hơn thực tế thiết kế của yêu cầu dự án đề ra. Vì vậy, chúng ta có thể làm đơn giản mẫu đó nhằm mang lại hiệu quả cao hơn.

| Đặc tính của lập trình hướng đối tượng (OOP)

- Tính đóng gói (**Encapsulation**)
- Tính kế thừa (**Inheritance**)
- Tính đa hình (**Polymorphism**)
- Tính trừu tượng (**Abstraction**)

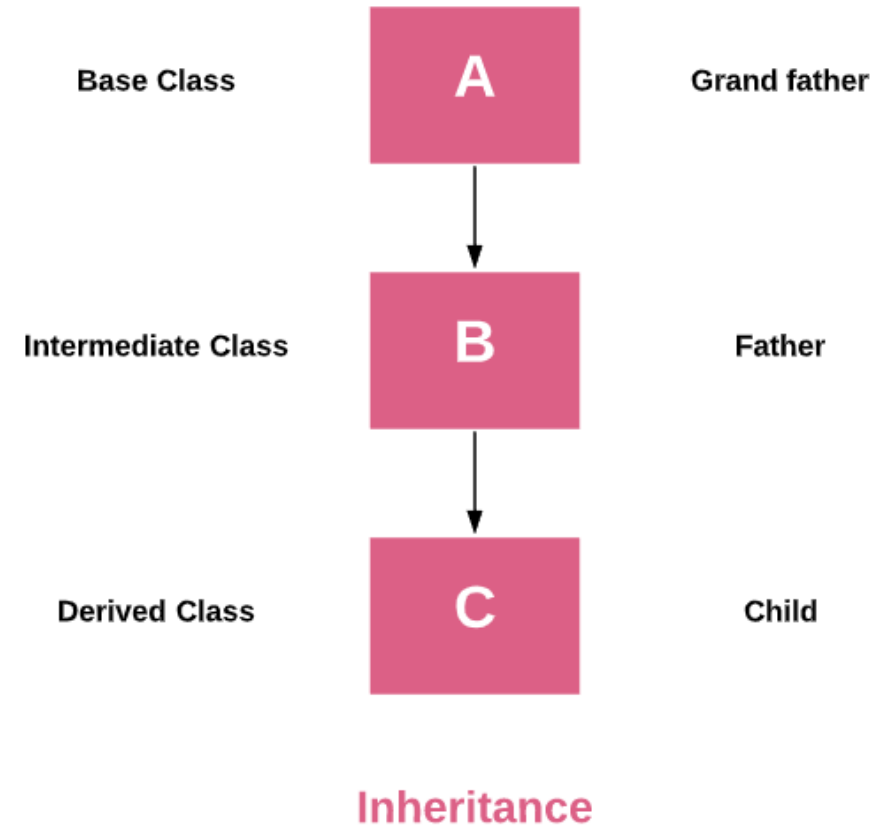
| Tính đóng gói (Encapsulation)

- Tính đóng gói được hiểu đơn giản các dữ liệu (data) và các hàm/phương thức (functions/methods) có liên quan đến nhau được “đóng gói” thành các lớp (class) để tiện cho việc quản lý và sử dụng.



| Tính kế thừa (Inheritance)

- Tính kế thừa trong lập trình hướng đối tượng thể hiện mối quan hệ **cha-con** giữa các lớp (classes) với nhau và nhờ vào mối quan hệ này cho phép lớp con thừa hưởng những đặc tính của lớp cha.



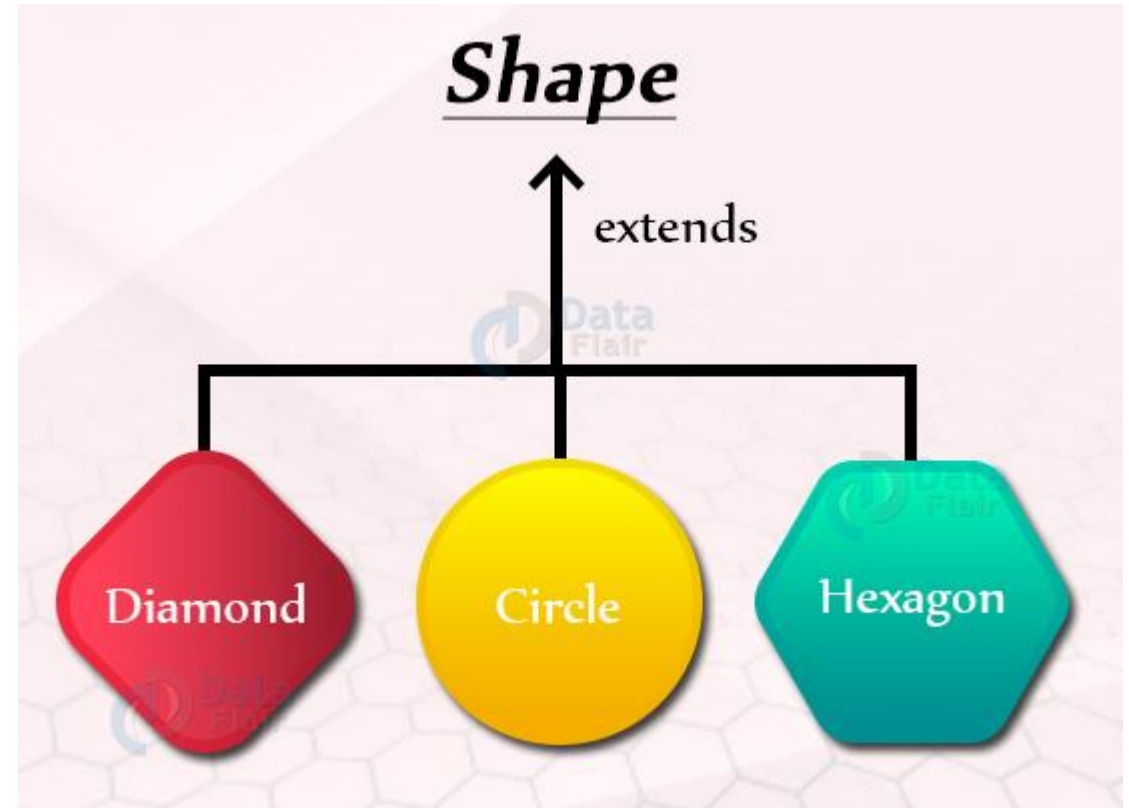
| Tính đa hình (Polymorphism)

- **Tính đa hình** (Polymorphism) là một đối tượng có thể thực hiện một tác vụ theo nhiều cách khác nhau.
- Nó cho phép thực hiện cùng một hành động trên các đối tượng khác nhau mà không cần quan tâm đến loại đối tượng cụ thể.
- Giúp tái sử dụng mã nguồn, linh hoạt và phân cấp, dễ bảo trì.



| Tính trừu tượng (Abstraction)

- **Tính trừu tượng** là khái niệm chỉ “hiển thị” các thuộc tính cần thiết và “ẩn” thông tin không cần thiết.
- Mục đích chính của trừu tượng là che giấu những chi tiết không cần thiết khỏi người dùng, giảm sự phức tạp của bài toán.
- Che dấu thông tin vì vậy đảm bảo được sự độc lập của từng phần của bài toán, chia bài toán thành từng phần trừu tượng khác nhau và giải quyết bài toán trên từng mức trừu tượng đó.



| 5.5 MẪU THIẾT KẾ CƠ BẢN: LỚP INTERFACE VÀ ABSTRACT

Abstraction là gì?

- Tính trừu tượng (**Abstraction**) trong Java là khả năng mô tả và tập trung vào các đặc tính quan trọng của một đối tượng, bỏ qua các chi tiết không quan trọng, và tạo ra các lớp, phương thức và thuộc tính trừu tượng để thể hiện các tính năng của đối tượng đó.
- Tính trừu tượng trong Java được thực hiện thông qua các cơ chế:
 - Lớp trừu tượng (abstract class)
 - Giao diện (Interface)

| 5.5 MẪU THIẾT KẾ CƠ BẢN: LỚP INTERFACE VÀ ABSTRACT

Abstract Class là gì?

- **Abstract class** là một lớp trừu tượng mà không thể được khởi tạo trực tiếp. Nó được sử dụng để định nghĩa các phương thức trừu tượng mà các lớp con cần triển khai.
- Nó chứa ít nhất một phương thức trừu tượng, tức là phương thức không có phần thân hàm, và được khai báo bằng từ khóa “**abstract**”.
- Các phương thức trừu tượng không có thân (body) và cần được khai báo trong các lớp con của abstract class. Các lớp con phải triển khai (implement) tất cả các phương thức trừu tượng của lớp cha.
- **Abstract class** thường được sử dụng để định nghĩa các lớp cơ sở (base classes) trong một hệ thống lớp phức tạp, và để đảm bảo tính đa hình (polymorphism) trong mã Java.

| Mục đích sử dụng của Abstract Class trong Java

- Lớp trừu tượng trong Java cung cấp một mẫu (**Template**) để triển khai quá trình trừu tượng dữ liệu bằng cách giấu đi chi tiết cài đặt. Nó cũng cung cấp cho người sử dụng cuối một mẫu mô tả các phương thức có sẵn.
- Trừu tượng hóa dữ liệu trong Java giúp giảm sự phụ thuộc (**Loose Coupling**) giữa các thành phần của chương trình, giúp tăng tính linh hoạt và dễ bảo trì.
- Sử dụng lớp trừu tượng cho phép tái sử dụng mã (**Code Reusability**) giúp tiết kiệm thời gian. Các phương thức trừu tượng có thể được triển khai lại trong các lớp con.
- Bằng cách sử dụng hỗ trợ của quá trình giải quyết phương thức động (**Dynamic Resolution**), lập trình viên có thể giải quyết nhiều vấn đề với sự trợ giúp của một phương thức trừu tượng.

| Lưu ý về lớp abstract trong Java

- Nó không thể được khởi tạo.
- Nó có thể có các phương thức trừu tượng và phương thức không trừu tượng.
- Nó có thể có các phương thức khởi tạo (**constructor**) và phương thức tĩnh (**static method**).
- Nó có thể có các phương thức cuối cùng (final) được dùng để buộc lớp con không được thay đổi nội dung của phương thức.

| Khi nào cần sử dụng Abstract Class?

- Cung cấp chức năng mặc định cho các lớp con.
- Cung cấp một mẫu (template) cho các lớp cụ thể trong tương lai.
- Giúp bạn định nghĩa một giao diện chung cho các lớp con của nó.
- Lớp trừu tượng cho phép tái sử dụng mã ([code reusability](#)).
- Khi ta muốn sử dụng tính kế thừa vì Abstract Class cung cấp một cài đặt lớp cơ sở chung cho các lớp được kế thừa.
- Nếu chúng ta muốn tạo nhiều phiên bản của thành phần. Bằng cách cập nhật lớp cơ sở, tất cả các lớp kế thừa sẽ tự động được cập nhật với thay đổi đó.
- Tăng khả năng tương thích chuyển tiếp (forward compatibility). Nếu sử dụng interface, chúng ta không thể thay đổi mà không làm hỏng mã nguồn nhưng nếu dùng lớp abstract class, chúng ta vẫn có thể thêm hành vi mà không làm hỏng mã nguồn hiện tại.

| Các thao tác cơ bản với Abstract Class

Khai báo Abstract Class trong Java

- Để khai báo một lớp trừu tượng trong Java, ta sử dụng từ khóa “abstract” và đặt nó trước tên lớp. Cú pháp:

```
<Access_Modifier> abstract class <Class_Name> {  
    //Data_Members;  
    //Statements;  
    //Methods;  
}
```

- Trong đó:
 - + **Access_Modifier**: từ khóa chỉ định quyền truy cập của lớp trừu tượng.
 - + **Abstract class**: từ khóa định nghĩa một lớp trừu tượng.
 - + **Class_Name**: tên lớp trừu tượng.
 - + **Data_Members**: các biến thành viên (member variables) của lớp trừu tượng
 - + **Statements**: các câu lệnh, chú thích hoặc bất kỳ nội dung nào khác bạn muốn đặt trong lớp trừu tượng.
 - + **Method**: phương thức của lớp trừu tượng có thể là phương thức trừu tượng (abstract method), phương thức không trừu tượng (non-abstract method) hoặc cả hai.

- Ví dụ:

```
public abstract class Animal {  
    public abstract void makeSound();  
}
```

| Các thao tác cơ bản với Abstract Class

Khai báo một phương thức trừu tượng

- Cú pháp:

abstract <modifier> <return_type> <method_name>(<parameter_list>);

- Trong đó:

- + **abstract**: từ khóa để khai báo phương thức trừu tượng.
- + **modifier**: các từ khóa truy cập.
- + **return_type**: kiểu dữ liệu của giá trị trả về của phương thức hoặc từ khóa void nếu phương thức không trả về giá trị nào.
- + **method_name**: tên phương thức.
- + **parameter_list**: danh sách các tham số của phương thức, được phân tách bằng dấu phẩy (,).

- Ví dụ:

public abstract void makeSound();

| Ghi đè phương thức trừu tượng

- Lớp con phải triển khai lại tất cả các phương thức trừu tượng của lớp cha hoặc định nghĩa lại lớp con là một lớp trừu tượng. Việc ghi đè phương thức trừu tượng cho phép các lớp con tùy chỉnh triển khai của phương thức trừu tượng theo nhu cầu của chúng.
- Khi lớp con triển khai lại một phương thức trừu tượng, chúng ta sử dụng từ khóa “**override**” để đánh dấu phương thức đó là phương thức được ghi đè. Cú pháp:

```
@Override  
<modifier> <return_type> <method_name>(<parameter_list>) {  
    // code implementation  
}
```

- Ví dụ:

```
@Override  
public void makeSound() {  
    System.out.println("Bark!");  
}
```

| Kế thừa từ một lớp trừu tượng

- Do lớp trừu tượng không thể tạo ra đối tượng hoặc triển khai trực tiếp từ lớp đó. Do đó, để truy cập các phương thức bên trong các lớp trừu tượng, chúng ta phải thực hiện kế thừa từ lớp trừu tượng đó bằng cách sử dụng từ khóa “**extends**” trong khai báo lớp con.
- Lớp con phải triển khai tất cả các phương thức trừu tượng có trong lớp cha trừ khi lớp con cũng là một lớp trừu tượng.

| Kế thừa từ một lớp trừu tượng

- Cú pháp:

```
abstract class TenLopCha {  
    // các phương thức trừu tượng  
}  
class TenLopCon extends TenLopCha {  
    // định nghĩa các phương thức trừu tượng và/hoặc phương thức khác  
}
```

- Ví dụ:

```
public class Dog extends Animal {  
    // code here  
}
```

| Interface trong Java

- Một **Interface** trong Java là một bản thiết kế của một lớp. Nó chỉ có các phương thức trừu tượng. Interface là một kỹ thuật để thu được tính trừu tượng hoàn toàn và đa kế thừa trong Java.
- **Interface** không chứa bất kỳ phương thức cụ thể nào (phương thức có mã) mà chỉ chứa các phương thức trừu tượng (abstract methods) và các hằng số (constants). Các phương thức trong interface không có thân (body) và chỉ định tên, tham số và kiểu trả về.
- **Interface** không phải là một đối tượng và không thể được khởi tạo. Tuy nhiên, các lớp triển khai (implement) interface có thể được khởi tạo.
- **Interface** không chứa các biến thể hiện (instance variables), nhưng nó có thể chứa các biến public static final (constants) để định nghĩa các giá trị hằng số cho các lớp triển khai.

| Đặc điểm của interface

- Không thể khởi tạo một interface.
- Một interface không chứa bất cứ hàm Constructor nào.
- Tất cả các phương thức của interface đều là abstract.
- Một interface không thể chứa một trường nào trừ các trường vừa static và final.
- Một interface không thể kế thừa từ lớp, nó được triển khai bởi một lớp.
- Một interface có thể kế thừa từ nhiều interface khác.

| Khi nào cần để sử dụng Interface?

- Để đạt được tính trừu tượng hoàn toàn.
- Để hỗ trợ giải quyết phương thức động tại thời điểm chạy.
- Giúp bạn đạt được kết nối lỏng (Loose Coupling).
- Cho phép bạn tách định nghĩa của một phương thức khởi thừa kế phân cấp.
- Muốn cung cấp chức năng (functionality) trên nhiều đối tượng khác biệt. Abstract class chỉ nên sử dụng chủ yếu cho các đối tượng có liên quan chặt chẽ, trong khi Interface phù hợp nhất để cung cấp chức năng chung cho các lớp không liên quan.
- Khi chúng ta muốn có một cái gì đó tương tự như kế thừa đa năng vì chúng ta có thể thực hiện nhiều giao diện.
- Khi ta thiết kế các đoạn chức năng nhỏ, ngắn gọn. Nếu chúng ta thiết kế các đoạn chức năng lớn hơn, hãy sử dụng lớp trừu tượng.

| Khai báo interface

- Cú pháp:

```
interface <tên interface> {  
    // Khai báo các thành phần bên trong interface  
}
```

- Ví dụ:

```
interface Printable {  
    void print();  
}  
class A6 implements Printable {  
    @Override  
    public void print() {  
        System.out.println("Hello");  
    }  
  
    public static void main(String args[]){  
        A6 obj = new A6();  
        obj.print();  
    }  
}
```

| Đa thừa kế trong Java bởi Interface

- Ví dụ một lớp cài đặt (implements) nhiều interface:

```
public interface Shape {  
    void draw();  
}  
public interface Color {  
    String getColor();  
}  
public class Rectangle implements Shape, Color {  
    @Override  
    public void draw() {  
        System.out.println("Draw " + this.getColor() + " rectangle");  
    }  
    @Override  
    public String getColor() {  
        return "red";  
    }  
}
```

| Đa thừa kế trong Java bởi Interface

- Ví dụ interface kế thừa (extend) nhiều interface:

```
public interface Shape {  
    void draw();  
}  
public interface Color {  
    String getColor();  
}  
public interface ShapeColor extends Shape, Color {  
}  
public class Circle implements ShapeColor {  
    @Override  
    public void draw() {  
        System.out.println("Draw " + this.getColor() + " circle");  
    }  
    @Override  
    public String getColor() {  
        return "red";  
    }  
}
```

| Interface lồng nhau

- Một Interface có thể có Interface khác, đó là lồng Interface:

```
interface Printable {  
    void print();  
  
    interface MessagePrintable {  
        void msg();  
    }  
}
```

| Sự khác nhau giữa Abstract class và Interface

Abstract class	Interface
1) Abstract class có phương thức abstract (không có thân hàm) và phương thức non-abstract (có thân hàm).	Interface chỉ có phương thức abstract. Từ java 8, nó có thêm các phương thức default và static.
2) Abstract class không hỗ trợ đa kế thừa.	Interface có hỗ trợ đa kế thừa
3) Abstract class có các biến final, non-final, static and non-static.	Interface chỉ có các biến static và final.
4) Abstract class có thể cung cấp nội dung cài đặt cho phương thức của interface.	Interface không thể cung cấp nội dung cài đặt cho phương thức của abstract class.
5) Từ khóa abstract được sử dụng để khai báo abstract class.	Từ khóa interface được sử dụng để khai báo interface.
6) Được trừu tượng 1 phần (0 tới 100%)	Được trừu tượng toàn phần (100%).
7) Ví dụ: <pre>public abstract class Shape { public abstract void draw(); }</pre>	Ví dụ: <pre>public interface Drawable { void draw(); }</pre>

Ví dụ về abstract class và interface trong java

```
1 //tạo interface có 4 phương thức
2 interface A {
3     void a();
4     abstract void b();
5     public void c();
6     public abstract void d();
7 }
8
9 // tạo abstract class cung cấp cài đặt cho một phương thức của interface A
10 abstract class B implements A {
11     public void c() {
12         System.out.println("I am C");
13     }
14 }
15
16 // tạo subclass của abstract class B, cung cấp cài đặt cho phương thức còn lại
17 class M extends B {
18     public void a() {
19         System.out.println("I am a");
20     }
21
22     public void b() {
23         System.out.println("I am b");
24     }
25
26     public void d() {
27         System.out.println("I am d");
28     }
29 }
30
31 // tạo lớp Test5 để gọi các phương thức của interface A
32 public class Test5 {
33     public static void main(String args[]) {
34         A a = new M();
35         a.a();
36         a.b();
37         a.c();
38         a.d();
39     }
40 }
```

Output:

```
I am a
I am b
I am C
I am d
```


| Bài tập

1. Sử dụng khái niệm lớp giao diện **interface** để xây dựng một ứng dụng tính và hiển thị lương của các loại nhân viên khác nhau trong một doanh nghiệp phần mềm cho trong Bảng sau. Yêu cầu cài đặt như sau: Dịch vụ tính lương cung cấp bởi các đối tượng thuộc loại khác nhau như CategoryA, CategoryB...được cài đặt từ **getSalary()** của interface **SalaryCaculator**; Lớp nhân viên **Employee** triển khai interface **SalaryCaculator** và cài đặt phương thức hiển thị **display()**.

- a) Vẽ biểu đồ UML để thể hiện các quan hệ trên.
- b) Cài đặt đầy đủ với lớp thực thi MainApp. Lương được tính theo công thức lương cơ bản cộng với lương làm thêm giờ.

| Bài tập

Bảng phân loại nhân viên

Nhân viên	Loại
Lập trình, thiết kế và tư vấn	Loại A, lương cơ bản 2000, tăng giờ 15/giờ
Đại diện bán hàng, quản lý bán hàng, kế toán, nhân viên kiểm chứng	Loại B, lương cơ bản 1500, tăng giờ 10/giờ
Nhân viên bán hàng, nhân viên tiếp thị	Loại C, lương cơ bản 800, tăng giờ 5/giờ

| Bài tập

2. Thiết kế **interface Search** khai báo phương thức tìm một mặt hàng như sách Book trong một danh sách. Thiết kế và cài đặt hai lớp tìm kiếm nhị phân **BinarySearch** và tìm kiếm tuần tự **LinearSearch** để thực thi các cách tìm kiếm trong danh sách.

3. Thiết kế interface **AddressValidator** khai báo các phương thức để xác nhận các phần khác nhau của một địa chỉ. Thiết kế và cài đặt hai lớp **USAAddress** (Hoa kỳ) và **VNAddress** (Việt nam) để xác nhận các địa chỉ của các nước tương ứng.

| Bài tập

4. Trong một doanh nghiệp, nhân viên thường được thể hiện dạng phân cấp lớp với lớp cơ sở là nhân viên **Employee** và sau đó là nhân viên đại diện bán hàng **SalesRep**, nhân viên tư vấn **Consultant**... Hãy tạo một số phương thức sau đây trong lớp **Employee**:

- Lưu và hiển thị dữ liệu nhân viên.
 - Truy nhập thuộc tính nhân viên như tên, id.
 - Tính thu nhập hàng tháng cho nhân viên.
- a) Sử dụng lớp trừu tượng **abstract** để cài đặt yêu cầu trên. Chú ý rằng 2 phương thức đầu là chung cho các loại nhân viên, nhưng tính thu nhập hàng tháng lại là khác nhau cho các loại nhân viên.
- b) Sử dụng lớp interface để cài đặt. So sánh hai cách cài đặt này.